

Safe Regression Test Suite Optimization: A Review

Aftab Ali Haider¹, Aamer Nadeem², Shamaila Akram³

Center for Software Dependability Mohammad Ali Jinnah University (MAJU), Islamabad, Pakistan

^{*}University of Calgary, UofC, Calgary, Canada

Email: aftab775@yahoo.com¹, anadeem@jinnah.edu.pk², shamaila_akram@yahoo.com³

Abstract—Systems are frequently regression tested for frequently occurring changes due to corrective, preventive, adaptive or perfective actions. Regression testing is used to prevent the undesired effect of these changes on the previously tested version. Due to these changes, new test cases become part of the test suite making it huge and inefficient for ‘retest all’ strategy. The ultimate solution of this problem is optimization or reduction of the test suite. Computational intelligence (CI) based approaches like evolutionary computation, fuzzy logic, neural networks and swarm optimization has been used for test suite reduction. Optimization approaches reduce the test suite by compromising its safety. Ideally optimization of test suite must guarantee safe reduction. In this work, **we have optimized the test suite using some CI based approaches and then analysed the test suite for ‘safe reduction’**. Safe reduction can be gauged using control flow graphs. Test cases of optimal solutions were traversed on these graphs. We found that these solutions partially cover control flow graph. This showed that optimal solutions returned by CI based approaches except fuzzy logic are not safe and will be inadequate for regression testing.

Keywords: Test Suite Optimization; Multi Objective Optimization; Computational Intelligence; Safety Parameters; and Safe Reduction

I. INTRODUCTION

Software reuse, rapid iterative development, integration testing and software maintenance require retesting the system with previously qualified test cases by the baseline version [1-2]. The purpose of retesting is to ensure that changes have not affected the already tested functionalities and changed part of the system behaves as intended [3]. This retesting activity is known as regression testing and is performed on delta build with baseline test cases [2]. Any adaptive, corrective, preventive or perfective action will require regression testing of the system [1]. Regression testing is a mandatory requirement whenever a change in previously tested system is made [2]. After each modification, size of regression test suite increases due to addition of new test cases for changed part of the system [4]. Regression testing is performed frequently due to frequent changes in the system requirements and it will be expensive to execute entire test suite again and again [3]. As an alternative, a subset of test cases as efficient as the original test suite may be used for regression testing [5]. This reduction in test suite using some heuristics with some predefined objectives is known as test suite optimization. Test suite optimization may be helpful to perform regression testing within constrained resources [4]. There exists a trade-off between the reduction of test suite and its effectiveness [2]. As

a consequence of optimization, effectiveness of test suite is deterred [3]. Besides the effectiveness of testing, the major concern that arises after test suite optimization is ‘safe reduction’ of test suite [2]. Safe optimization requires that all those test cases that find faults for changed requirements are included and all those test cases that can not reveal regression faults are not included in the optimized test suite [2]. Any testing that does not include test cases for changed part is insufficient testing and is unsafe and may be attributed as inadequate testing [6]. ‘Retest all’ is the safest regression testing strategy and requires that system is retested with each test case in the test suite [2]. Ideally ‘retest all’ is the best strategy but is costly and time consuming [6]. Changes in the previously tested system may affect some test cases in the test suite by rendering these as broken, obsolete or redundant test cases [5]. It is useless to run these test cases and has no value addition to the testing [6]. These test cases are undue burden and must be eliminated from test suite [4]. A drawback of ‘retest all’ strategy is that it will run these ‘not required’ test cases and will not be efficient [2]. Some of the regression testing strategies consider dependency analysis and analysis of code for the changed part of the system. **Since these strategies require analysis of the changed part of the system to select the test cases for representative subset, therefore, these are considered safe reduction strategies. Other strategies that retest by profile or by risky use cases are unsafe strategies.** This work is concerned with few research questions related to test suite optimization.

- a. RQ1: Can single objective optimization reveal ideal representative subset for optimization.
- b. RQ2: Can safe optimization be guaranteed without careful selection of software safety parameters.
- c. RQ3: Do computational intelligence based techniques ensure safe optimization.

Mostly, optimization approaches intend to reduce the size of test suite without ensuring safety parameters as optimization objectives. This work will also identify those optimization parameters that must be ‘at least’ considered for safe optimization [7-8].

Rest of the paper is arranged such that section II reviews literature on test suite optimization approaches based on computational intelligence. Section III, briefly identifies importance of safety parameters. Section IV is about analysis of reviewed literature and identifies the research gap in existing optimization approaches, Section V is last but not the least and concludes this research work and opens fronts for

future research work. optimize test suite with the exception of Artificial Immune System [6, 8]. These approaches have been successfully used to optimize the test suite by reducing test suite size. [8].

Suri et al (2011a) [9] and Kaur and Goyal (2011a) [10] used Bee Colony Optimization (BCO) to reduce the test suite. They considered 'all faults coverage' and 'all path coverage' in minimum execution time respectively. Although fault coverage and path coverage provide safe reduction yet there is need to use both parameters complementally for safe optimization. Both these researchers used fault coverage and path coverage in isolation. Their work was intended to reduce the size without consideration of safe reduction.

Suri and Singhal (2011) [11, 12] and Vivekanandan et al. (2012) [13] used Ant Colony Optimization (ACO) for test suite reduction. They have used 'path coverage in minimum time' and 'find faults earlier or in short time' as optimization objectives respectively. Path coverage in minimum time may choose test cases that are unsuccessful to optimize other objectives e.g. finding all faults. Similarly finding faults earlier may not traverse all paths. Although all path coverage can provide safe reduction but it lacks safe optimization. Safe reduction of test suite does not ensure optimization of test suite but safe optimization ensures safe reduction and optimization.

Sehrawat et al (2012) [14], Kaur and Goyal (2011b & 2011c) [15-16], Krishnamoorthi et al. (2009) [17], Nachiyappan et al. (2010) [18] and Jyoti and Bhambhu (2015) [19] used Genetic Algorithms (GA) for test suite optimization. These work used 'entire path coverage', 'all independent path coverage', 'fault detection', maximal code coverage, coverage and 'execution time based fitness' and 'all fault coverage' as optimization objectives. They have been successful to reduce the test suite size but these results can be further improved through implementation of multiple objectives and safe optimization approach. Jayaparaksh and Alagarsamy (2015) [20] used distinctive genetic algorithm that provides complete branch coverage for test suite optimization. This approach was successful to reduce size and execution time of test suites. Malhotra and Khari (2014) [21] used mutated bee colony optimization algorithm to select all those test cases that provide distinct path coverage. These test cases are then mutated to obtain resultant test suite with optimized coverage.

Yoo and Harman [22] used Multi Objective Genetic Algorithms for test suite optimization and compared additional greedy algorithm with NSGA-II using code coverage, fault detection and execution cost as optimization objectives. Ma et al, [23] proposed multi-objective genetic algorithm for test suite reduction. In their work, they selected coverage and cost as optimization objectives. They inferred that multi-objective genetic algorithms are cost effective and are suitable for test suite reduction. Although this approach considered safety parameters, yet it did not focus on safe reduction of test suite.

Particle Swarm Optimization (PSO) approach has been used by Kaur and Bhatt (2011a & 2011b) [24-25] and Suri et al. (2012) [26] in combination with Genetic Algorithms for test suite optimization. It is good to benefit from GA and PSO in hybrid optimization approaches but validity of resultant test suite as

'safe and best suitable' remains unaddressed. Souza et al used Multi-objective Particle Swarm Optimization

(MOPSO) and its variants with functional requirements and cost effort as optimization objectives for test suite optimization [27]. Although author has used multiple objectives for optimization but has not considered safe optimization to obtain optimal test suite.

Sehrawat et al. [14] used Neural Networks (NN) in combination with genetic algorithms for test suite optimization. Author selected those test cases for mutation that provide all path coverage. Then using the concept of neurons for mutation, only those test cases are selected that find more faults in the same time and give more path coverage. The approach used multiple objectives but lacks safe optimization of test suite. Simao et al. [2006] used neural networks to classify test cases and determine feature vectors in clusters. To select the most suitable test cases for changed code, new software version is analyzed and most suitable cluster of test cases are selected [28].

Fuzzy expert systems have been proposed for optimization of test suite [29-32]. Xu et al. [29] proposed an expert system for test case selection in regression testing. They generated casual model using different parameters and then developed rules from this casual model. Malz et al. [30] discussed automated prioritization of test cases to ease the test managers having constraints of time and resources. Their work concentrated on use of software agents with fuzzy logic. Agents were used to determine the importance of test cases based on requirement changes. Bhasin et al. [31] has proposed fuzzy expert system for regression testing to prioritize test cases. Author has proposed test case evaluator, local and global priority estimator and software module importance estimator as input for fuzzy inference in decision making to prioritize test cases. Fuzzy clustering approach used to classify the similar test cases for test suite selection has been proposed by Kumar et al. [32]. Clustering helped to identify the redundant and similar test cases and was helpful to reduce the test suite. If the classes for changed part are generated, then it will be more suitable to provide safe optimization of test suite. Secondly, finding the similarity among test cases can be made more accurate using multiple objectives.

Besides expert systems, fuzzy logic has been used to identify the suitable set of test cases for pre-defined set of optimization objectives [33-34]. Author presented a framework to select test cases and evaluate the fitness using fuzzy logic. Using fuzzy logic, author has generated fuzzy evaluation matrix for regression testing optimization parameters discussed in [33]. Fuzzy Logic is used to deal with uncertainties and Kumar et al. (2011) has justified the fuzziness involved in software testing and significance of using fuzzy logic for test suite optimization [34]. Haider et al. [6] proposed fuzzy and iterative fuzzy for test suite optimization and compared the results with contemporary evolutionary approaches. Rai and Tyagi [35] proposed hybrid algorithm using fuzzy inference system and bee colony optimization. It uses bee colony optimization to determine the faults covered and execution time for each test case and later priority of each test case is assigned using fuzzy inference system. Joseph and Radhamani [36] proposed integration of fuzzy logic with glass-worm swarm algorithm

and optimized test cases for high priority node in regression testing. Recently, Anwar and Ahsan [37] had used neuro-fuzzy logic for test suite optimization. Comparative analysis of neuro-fuzzy logic with multi objective evolutionary algorithms shows superiority of fuzzy logic.

II. SAFE REDUCTION

Whether safe reduction can be achieved through optimization of single objective? Safe reduction of test suite can be defined such that all those test cases that can reveal faults are included and all those test cases that cannot reveal faults are not included in the resultant test suite. Original test suite comprises all those test cases that can or cannot reveal faults during regression testing. Original test suite is represented in Eq.1

$$T = T_f \cup \neg T_f \quad \text{Eq-1}$$

Where,

T = Original Test Suite

T_f = Fault Revealing Test Cases

$\neg T_f$ = Not Fault Revealing Test Cases

An optimized test suite (S) is subset of original test suite (T) and includes all fault revealing test cases.

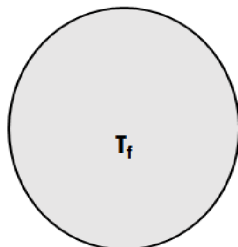
Safely Reduced Test Suite (S) = T_f

Un-Safely Reduced Test Suite (S') =

All or some T_f AND all or some $\neg T_f$

Suppose a Single safety parameter, coverage criterion, has been used for test suite optimization without consideration of fault detection. Is it possible to include all faults revealing test cases in optimized test suite with coverage optimization? Four possibilities may result;

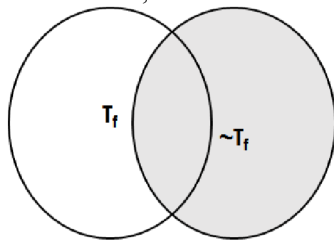
Possibility-1: All those test cases that can find regression faults are included and all those test cases that cannot find regression faults are not included given in Fig 1. In the resultant test suite, Precision and Inclusiveness is achieved.



All T_f and None $\neg T_f$

Fig.1. Scenario1, All T_f and None $\neg T_f$

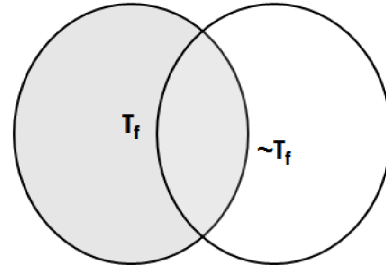
Possibility-2: Some of the test cases that can reveal regression faults are not included in the test suite given in Fig. 2. Resultant test suite is Precise, but Inclusiveness is not achieved.



Some T_f and All $\neg T_f$

Fig.2. Scenario 2, Some T_f and All $\neg T_f$

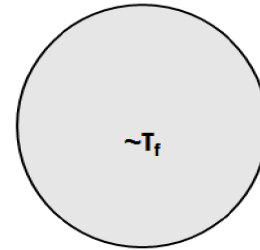
Possibility-3: Some of the test cases that can not reveal regression faults are included in the test suite given in Fig. 3. It might be Inclusive but it is not precise test suite.



All T_f and Some $\neg T_f$

Fig.3. Scenario 3, All T_f and Some $\neg T_f$

Possibility 4: All those test cases that can reveal regression faults are not included and all those test cases that cannot reveal regression faults are included given in Fig 4. Resultant test suite is neither Precise nor Inclusive.



None T_f and All $\neg T_f$

Fig.4. Scenario 4, None T_f and Some $\neg T_f$

If there exist “n” test cases that find more than one faults on some path e.g. ABCDFGIJLN given in Fig. 5, then any optimization algorithm will select some of these test cases. Although all those test cases that optimize coverage criterion are included but some of the critical test cases are not included in resultant test suite. This situation will result in possibility-2. All possibilities except possibility-1 are unsafe.

Similarly, if fault detection is selected as single safety parameter for optimization, safe reduction will not be achieved. There might be a scenario such that test cases ‘n’ can find only 1 fault or none on path ABCDFGIJLN. There might be many faults on other paths of the control flow graph. Interestingly, path ABCDFGIJLN is either modified path or it contains critical fault. Optimization algorithm using fault detection as safety parameter may or may not select test case for path ABCDFGIJLN. Although resultant test suite has been optimized for fault detection but it does not contain an important test case. This scenario will again result in possibility-2.

This work concludes that single objective optimization will never result in safe reduction of test suite. Selection of multiple objectives with inclusion of fault detection coverage parameters may result in safe reduction. Although, it has been concluded that single objective can never guarantee safe reduction, yet research initiative is required to determine “Whether multiple objective optimization may result in safe reduction?”

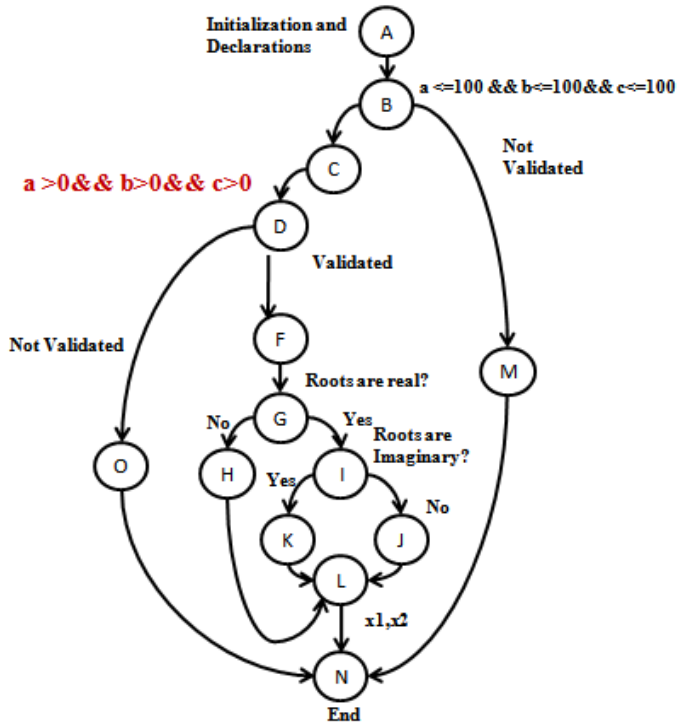


Fig.5. Sample control flow graph

III. ANALYSIS OF LITERATURE REVIEWED

Reviewed literature elaborated in Table 1, can be classified into four main categories for better understanding of the related work. These four categories are;

1. Research work using single objective optimization and considering safety parameters
2. Research work using single objective optimization and does not consider safety parameters
3. Research work using multiple objectives and does not consider safety parameters.
4. Research work with multiple objectives optimization and using safety parameters.

It is evident in Fig 6, that most of the single objective techniques consider safety parameters for test suite optimization. However, safety parameters are not being given due importance in multi objective optimization. Almost 85 percent approaches that used single optimization objective have considered safety parameters during optimization process, whereas, only 50 percent multi objective approaches have used safety parameters for optimization purpose. This research has concluded in section 3 that safe reduction is not possible using single safety parameter. It can also be concluded from reviewed literature that safe reduction has not been considered in almost 73 percent of research work in the domain of test suite optimization

Another, important fact concluded from reviewed literature is that safety parameters, fault coverage and path coverage are most commonly chosen parameter for test suite reduction given in Fig.8. However, there is need to understand that both these parameters must be complimentary used for safe reduction.

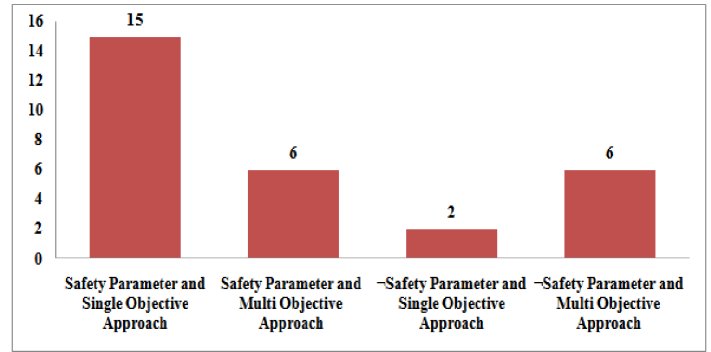


Fig.6. Safety parameters and reviewed literature

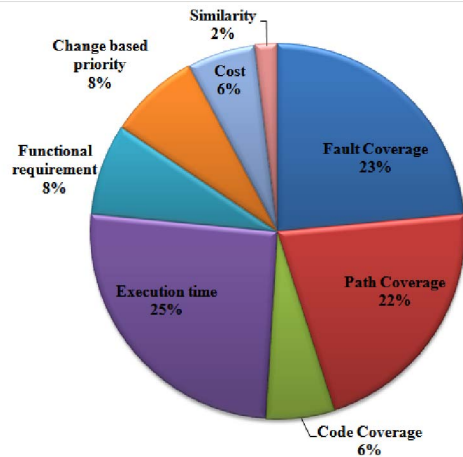


Fig.7. Distribution of safety parameters in reviewed literature

Computational intelligence approaches have been extensively used for test suite optimization as shown in Fig.9. An importance fact about these approaches is that these must be constrained for safe reduction. Safe selection approach proposed in [7] can be considered for these approaches.

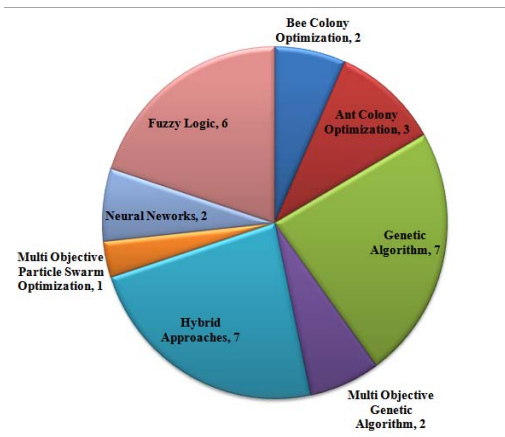


Fig.6. Computational intelligence approaches in reviewed literature

I. CONCLUSION AND FUTURE WORK

This research work concludes that regression test suite optimization can be rendered safe through consideration of safety parameters during optimization process. It has also been concluded that fault and path coverage are two primary safety

parameters for safe reduction. However, there may be other parameters derived from these basic parameters. It has also been inferred that single objective optimization even using basic safety parameters will never be safe. There is need to consider both of these parameters during optimization process. It has also been concluded that safe reduction has not been implemented in its true essence for computational intelligence bases optimization approaches.

Future research will determine, whether multiple objective optimization with safety parameters results in safe reduction? It will endeavor to provide some generic framework for safe reduction for computational based optimization approaches.

Table 1: Reviewed literature

Author	Algorithm	Safety Parameters	Objective(s)
Suri et al (2011a) [9]	Bee Colony Optimization	Yes	Single
Kaur and Goyal(2011a) [10]		Yes	Single
Suri and Singhal (2011) [11-12]	Ant Colony Optimization	Yes	Single
Vivekandan et al (2012) [13]		Yes	Single
Sehrawat et al (2012) [14]	Genetic Algorithm	Yes	Single
Kaur and Goyal(2011b) [15]		Yes	Single
Kaur and Goyal (2011c) [16]		Yes	Single
Krishnamoorthi et al. (2009) [17]		Yes	Single
Nachiyappan et al. [18]		Yes	Single
Jyoti and Bhambhu [19]		Yes	Single
Jayaparaksh and Alagarsamy [20]		Yes	Single
Yoo and Harman [22]	Multi Objective Genetic Algorithm	Yes	Multiple
Ma et al [23]		Yes	Multiple
Kaur and Bhat (2011a) [24]	Hybrid Approach (GA and PSO)	Yes	Single
Kaur and Bhat (2011b) [25]		No	Single
Suri et al (2012) [26]		No	Single
Malhotra and Khari [21]	Mutated Bee Colony Optimization	Yes	Single
Rai and Tyagi [35]	BCO and fuzzy Logic	Yes	single
Souza et al [27]	Multi-objective Particle Swarm Optimization	No	Multiple
Sehrawat et al [14]	Neural Networks	Yes	Multiple
Simao et al [28]		No	Multiple
Anwar and Ahsan [37]	Neuro Fuzzy Expert system	Yes	Multiple
Xu et al [29]	Fuzzy Expert System	No	Multiple
Malz et al [30]		No	Multiple
Bhasin et al [31]		No	Multiple
Kumar et al [32]		No	Multiple
Haider et al [4]	Fuzzy Inference system	Yes	Multiple
Haider et al [8]	Iterative Fuzzy Expert system	Yes	Multiple
Joseph and Radhamani [36]	Fuzzy Logic and Glassworm Swarm Algorithm	Yes	Multiple

REFERENCES

- [1] R. Pressman, 2010. Software Engineering: A Practitioner's Approach, 7th ed., McGraw Hill.
- [2] R. Binder, 2004. Testing Object Oriented Systems: Models, Patterns, and Tools, 5th ed., Addison Wesley.
- [3] Kadri, S. 2011. A New Proposed Technique to Improve Software Regression Testing Cost. *International Journal of Security and Its Applications*, 5, 3 (Jul. 2011).
- [4] A.A. Haider,., Rafiq, S., Nadeem, A. 2012. Test suite optimization using fuzzy logic. In *Proceedings of 8th International Conference on Emerging Technologies* (Islamabad, Pakistan, Oct 2012). doi: 10.1109/ICET.2012.6375440
- [5] A.A Haider, Nadeem, A. Rafiq, S. 2013. On the fly Test Suite Optimization with FuzzyOptimizer. In *Proceedings of Frontiers of Information Technology* ((Islamabad, Pakistan, Dec 2013).
- [6] A.A. Haider. A. Nadeem, S. Rafiq2014. Multiple Objective Test Suite Optimization: A Fuzzy Logic Based Approach. *Journal of Intelligent and Fuzzy Systems*. 27, 2, 863-875. DOI: 10.3233/IFS-131045.
- [7] Rothermal G. 1997. *A Safe, Efficient Regression Selection Technique*. ACM Transactions on software engineering and methodology, 6, 2 (Apr. 1997), 173-210.
- [8] Haider, A.A., Rafiq, S., Nadeem, A. Computational Intelligence and Safe Reduction of Test Suite, In *Proceedings of 9th International Conference on Emerging Technologies, Pakistan, Dec-2013*.
- [9] Suri, B., Mangal, I., Srivastava, V. Regression Test Suite Reduction using an Hybrid Technique Based on BCO and Genetic Algorithm. *Special Issue of International Journal of Computer Science & Informatics*, 2, 1, 165-172.
- [10] Kaur, A. Goyal, S. 2011. A Bee Colony Optimization Algorithm for Fault Coverage Based Regression Test Suite Prioritization. *International Journal of Advanced Science and Technology*, 29, 17-21.
- [11] Suri, b. Singhal, S. 2011. Implementing Ant Colony Optimization for Test Case Selection and Prioritization. *International Journal on Computer Science and Engineering*, 3, 5, 1924-1932.
- [12] Suri,B. Singhal, S. 2011. Implementing Ant Colony Optimization for Test Case Selection and Prioritization. *International Journal on Computer Science and Its Applications*, 286-291.
- [13] Vivekanandan, K. Lakshmi, G. 2012. Improving Regression Testing through Modified Ant Colony Algorithm on a Dependency Injected Test Pattern. *International Journal of Computer Science Issues*, 9, 2, 593-598.
- [14] Sehrawat, R., Srivastava, V., Mangal, I. 2012. A Neuro-Genetic Algorithm for Selection of Test Suites in Regression testing. *International Journal of Computer Application*, 2, 1.
- [15] Kaur, A. Goyal, S. 2011. A Genetic Algorithm for Regression Test Case Coverage Prioritization Using Code Coverage. *International Journal on Computer Science and Engineering*, 3, 5, 1839~1847.
- [16] Kaur, A., Goyal, S.2011. A Genetic Algorithm for Fault based Regression Test Case Prioritization. *International Journal of Computer Applications*, 32, 8.

- [17] Krishnamoorthi R., S.A.Sahaaya, A., Mary A.2009. Regression Test Suite Prioritization using Genetic Algorithms. *International Journal of Hybrid Information Technology*, 2, 3.
- [18] Nachiyappan, S. Vimaladevi, A. SelvaLakshmi, C. 2010. An Evolutionary Algorithm for Regression Test Suite Reduction. In *Proceedings of the International Conference on Communication and Computational Intelligence* (India, Dec. 2010), 503-508.
- [19] Jyoti, Bhambhu L. 2015. An Efficient Genetic Algorithm for Fault Based Regression Test Case Prioritization. *International Journal of Computer Engineering and Electronics Technology*, 1, 1, 1-5.
- [20] Jayaparakash, Alagarsamy, 2015. A distinctive genetic approach for test suite optimization, In *Proceedings of International Conference on Soft Computing and Software Engineering*, 62, 427-434, doi:10.1016/j.procs.2015.08.437
- [21] Malhotra R., Khari, M. 2014. Test Suite Optimization using Mutated Artificial Bee Colony, In *Proceedings of International Conference on Advances in Communication, Network, and Computing*, India, 45-54.
- [22] Yoo, S., Harman, M. 2007. Pareto Efficient Multi-Objective Test Case Selection. In *Proceedings of International Symposium on Software Testing and Analysis (ISSTA, 2007) London*, 140-150.
- [23] Ma, X. Sheng, B.,Ye, C. Test-Suite Reduction Using Genetic Algorithm. In *Proceedings of Advanced Parallel Processing Technologies (APPT, 2005)*, Hong Kong, 253-262.
- [24] Kaur, A. Bhatt, D. 2011. Particle Swarm Optimization with Cross-Over Operator for Prioritization in Regression Testing. *International Journal of Computer Applications*, 27, 10.
- [25] Kaur, A. Bhatt, D. 2011. Hybrid Particle Swarm Optimization for Regression Testing. *International Journal on Computer Science and Engineering*, 3, 5, 1815-1824.
- [26] Suri, B. Mangal, I. Analyzing Test Case Selection using Proposed Hybrid Technique based on BCO and Genetic Algorithm and a Comparison with ACO. *International Journal of Advanced Research in Computer Science and Software Engineering*, 2, 4.
- [27] Souza, L., de Miranda, P., Prudencio, R. Barros, F. 2011. A Multi-Objective Particle Swarm Optimization for Test Case Selection Based on Functional Requirements Coverage and Execution Effort. In *Proceedings of 23rd International Conference on Tools with Artificial Intelligence*, Bocca Raton, FL, 245-252. doi: 10.1109/ICTAI.2011.45.
- [28] Simao, A., Mellor, R., Senger, L. 2006. A Technique to Reduce the Test Case Suites for Regression Testing Based on a Self-Organizing Neural Network Architecture. In *Proceedings of 30th International computer software and application conference*. Chicago,
- [29] Xu, Z., K. Gao, K., Khoshgoftaar, T. 2005. Application of fuzzy expert system in test case selection for system regression test. In *Proceedings of Information Reuse and Integration (IRI 2005)*, 120-125. doi: 10.1109/IRI-05.2005.1506460
- [30] C. Malz, C., Jazdi, N., Gohner, P. 2012. Prioritization of test cases using software agents and fuzzy logic. In *Proceedings of Fifth International Conference on Software Testing, Verification and Validation*.
- [31] Bhasin, H., Gupta, S., Kathuria, M. 2013. Regression Testing using Fuzzy Logic. *International Journal of Computer Science and Information Technology*, 4, 2, 378-380.
- [32] Kumar, G., Kumar, P. 2013. Software testing optimization through test suite reduction using fuzzy clustering, *CSI Transactions in ICT*, 1, 3, 253-260.
- [33] Kumar, M. Sharma, A., Kumar, R. 2011. Towards Multi-Faceted Test Cases Optimization. *Journal of Software Engineering and Applications*, 4, 550-557.
- [34] Kumar, M. Sharma, A., Kumar, R. 2012. Multi faceted measurement framework for test case classification and fitness evaluation using fuzzy logic based approach, *Chiang Mai Journal of Science*, 39, 3.
- [35] Rai D., Tyagi, K. 2014. Regression Test Case Optimization Using Honey Bee Mating Optimization Algorithm with Fuzzy Rule Base, *World Applied Sciences Journal* 31 (4): 654-662. doi: 10.5829/idosi.wasj.2014.31.04.1336
- [36] Abraham, J., Radhamani, K. 2014. Optimization of Test Cases using Fuzzy based Glowworm Swarm Algorithm, *International Journal on Recent Trends in Engineering & Technology* 11, 2 54-62.
- [37] Anwar Z., Ahsan A. 2013. Comparative analysis of MOGA, NSGA-II and MOPSO for regression test suite optimization, *International Journal of Software Engineering*, 7, 1, 41-56.